

15. RL MDPs Q-Learning

Overview

Reinforcement learning formalizes decision making as Markov Decision Processes. We define returns, value functions, Bellman equations, and tabular Q-learning with ϵ -greedy exploration. This is the base for deep Q-networks in the next module.

Lab: Lab 15.

Contents

1	Sequential Decision Making	2
2	Markov Decision Process	2
3	Value Functions	2
4	Tabular Q-Learning	2
5	Exploration vs. Exploitation	2
6	Policy and Value Iteration	3
7	Lab	3
8	SARSA and On-Policy Control	3
8.1	Eligibility Traces	3
9	Extended Intuition	3
10	Numerical Walkthrough	4
11	PyTorch Implementation Notes	4
12	Local GPU Constraints	4
13	Debugging Checklist	4
13.1	MDP Design Choices Engineers Actually Control	5
14	Practice Problems	5

Module track

This module starts a new track. Later modules refer back using **Module NN** labels.

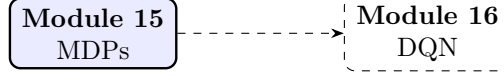


Figure 1: Module sequence (solid box: this PDF; dashed: typical next step).

1 Sequential Decision Making

Reinforcement learning [2] optimizes cumulative reward in environments with state transitions. Differs from supervised pipelines in **Module 09**.

2 Markov Decision Process

MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

$$P(s_{t+1} \mid s_t, a_t), \quad r_t = R(s_t, a_t), \quad \gamma \in [0, 1]. \quad (1)$$

Markov property: $P(s_{t+1} \mid s_t, a_t, s_{t-1}, \dots) = P(s_{t+1} \mid s_t, a_t)$.

Note. γ discounts future rewards; $\gamma \rightarrow 0$ myopic, $\gamma \rightarrow 1$ far-sighted (requires episodic termination).

3 Value Functions

State-value and action-value:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t = s \right], \quad (2)$$

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t = s, a_t = a \right]. \quad (3)$$

Optimal Q^* satisfies Bellman optimality:

$$Q^*(s, a) = \mathbb{E} \left[R + \gamma \max_{a'} Q^*(s', a') \mid s, a \right]. \quad (4)$$

4 Tabular Q-Learning

Model-free update for discrete $\mathcal{S}, \mathcal{A}$:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]. \quad (5)$$

Off-policy: behavior may be ϵ -greedy while learning greedy target.

5 Exploration vs. Exploitation

ϵ -greedy: choose random action with prob. ϵ , else $\arg \max_a Q(s, a)$. Decay ϵ from 1.0 to 0.01 over training.

Table 1: RL algorithm taxonomy.

Method	Policy	Value
Q-learning	Implicit (greedy)	Tabular Q
DQN (Module 16 (DQN loss))	Implicit	Neural Q
Policy gradient (Module 17 (policy gradient theorem))	Explicit π_θ	Optional

6 Policy and Value Iteration

Dynamic programming backups when P is known:

$$V_{k+1}(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V_k(s')]. \quad (6)$$

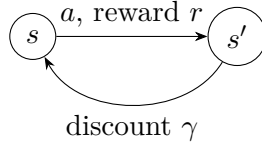


Figure 2: Agent-environment interaction loop.

7 Lab

Lab 15 implements tabular Q-learning on GridWorld using (5). Neural extension in **Module 16** (DQN loss).

8 SARSA and On-Policy Control

SARSA update uses actual next action a' from behavior policy:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)]. \quad (7)$$

Safer under exploration than Q-learning when ϵ -greedy explores dangerous states.

8.1 Eligibility Traces

TD(λ) blends n -step returns: $G_t^{(\lambda)} = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$.

9 Extended Intuition

Reinforcement learning models sequential decision making: an agent observes state s_t , takes action a_t , receives reward r_{t+1} , transitions to s_{t+1} . Markov Decision Processes assume the future depends on the present state only, not the full history (if state is sufficient). The return $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$ discounts future rewards with $\gamma \in [0, 1)$.

Value functions estimate expected return: $V^\pi(s)$ under policy π , or action-value $Q^\pi(s, a)$. Optimal Q^* satisfies the Bellman optimality equation:

$$Q^*(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right]. \quad (8)$$

Tabular Q-learning updates with learning rate α :

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]. \quad (9)$$

ϵ -greedy exploration trades random actions (explore) with greedy actions (exploit).

Deep methods in **Module 16** and **Module 17** scale this to neural function approximators; start tabular to verify the math.

10 Numerical Walkthrough

GridWorld 4×4 , $\gamma = 0.9$, $\alpha = 0.5$, $\epsilon = 0.1$, goal at bottom-right reward $+1$, step cost -0.04 .

Start $Q(s, a) = 0$ everywhere. From state s (row 0, col 0), action right, land in s' with $r = -0.04$:

$$Q(s, \text{right}) \leftarrow 0 + 0.5 \left(-0.04 + 0.9 \max_{a'} Q(s', a') - 0 \right) = 0.5(-0.04) = -0.02. \quad (10)$$

After many episodes, Q near goal states rises toward $1 - 0.04/(1 - \gamma)$ scaled by path length.

Episode length maybe 20 to 50 steps early, dropping toward optimal 6 steps as policy improves. With $\epsilon = 0.1$, roughly 10% random moves persist even after convergence (continued exploration).

1000 episodes on 4×4 : tabular update $\approx 15\text{k}$ steps total; runs in milliseconds on CPU.

11 PyTorch Implementation Notes

Tabular lab uses NumPy dict or `np.zeros((n_states, n_actions))`; no GPU needed.

- State indexing: flatten (*row, col*) to single index for table lookup.
- `np.argmax(Q[s])` for greedy action; `random` branch with prob ϵ .
- Log episode return `sum(rewards)` and steps per episode for learning curves.
- For later DQN (**Module 16**): same env API (`gymnasium`) eases transition.
- Reproducibility: `env.reset(seed=...)` and `np.random.seed`.

```
q = np.zeros((n_states, n_actions))
td_target = r + gamma * q[s_next].max()
q[s, a] += alpha * (td_target - q[s, a])
```

12 Local GPU Constraints

Tabular Q-learning is CPU-only; no VRAM concerns. If prototyping neural Q on the same GridWorld in **Module 16**, tiny MLP fits any GPU.

Use vectorized env wrappers only when moving to DQN at scale; tabular code stays readable with explicit loops.

13 Debugging Checklist

- Q values diverge to large magnitude: α too high or missing terminal state bootstrap ($\max Q = 0$ at goal).
- Policy never reaches goal: ϵ too low early or rewards too sparse; increase exploration or shape rewards.

- Oscillating policy: non-stationary if updating while acting on stale Q without proper episode boundaries.
- Same action always: tie-breaking not random in `argmax` on equal Q values.
- Off-by-one in s' after action: coordinate system does not match env `step` return.

13.1 MDP Design Choices Engineers Actually Control

The MDP is not handed to you by the environment API; you define what enters the state vector, how rewards are scaled, and when episodes terminate. A sparse goal reward (+1 at target) teaches slowly because most transitions look identical until the agent stumbles on success. Reward shaping adds dense signals (distance-to-goal penalty) but can change the optimal policy if shaping rewards are not potential-based; log both shaped and sparse returns when experimenting. Continuing tasks (robot locomotion) use undiscounted or γ very close to 1; episodic tasks (GridWorld, games) terminate and bootstrap $Q(s', a') = 0$ at terminal states.

Partial observability appears when the agent sees less than the full Markov state (frame pixels without velocity). Wrapping history into the state (stack last k observations) restores Markov property at the cost of larger tables or networks later in **Module 16**. Action space design matters: factorizing movement into 4 directions keeps the tabular grid tractable; continuous torque control needs policy methods from **Module 17**.

Policy evaluation computes expected return under a fixed policy; policy improvement takes $\arg \max_a Q(s, a)$ and iterates toward π^* . Q-learning is off-policy: it learns about the greedy policy while following ϵ -greedy behavior, which is why exploration schedule remains important even after Q looks converged. Discount $\gamma = 0.99$ vs 0.9 changes how many steps ahead the agent plans; on GridWorld, lower γ favors shorter paths even if per-step penalty is small. Always log episode length, total return, and visit counts per state when debugging; a policy that oscillates between two cells often signals incorrect terminal handling or reward sign errors. Seed environments and RNG for reproducible learning curves before comparing α or ϵ schedules across runs. Windy GridWorld variants (random action with prob 0.2 regardless of policy) teach robustness testing: optimal tabular policy should still reach goal in expectation. Sutton example 6.6 style: if step cost is -1 and goal reward is 0, optimal policy minimizes steps; sign errors in reward definition flip the learned path completely. Monte Carlo methods wait until episode end to update; TD methods bootstrap after one step. Tabular Q-learning is TD(0); compare update frequency when debugging slow convergence. State aggregation (grouping 4×4 grid into 2×2 macro cells) reduces table size but loses Markov property if critical detail sits inside a macro cell. Frozen policy evaluation: after Q converges, sweep $\epsilon = 0$ for 100 episodes and report mean return; compare to random policy baseline to quantify improvement. Stochastic transitions (slippery grid with 0.8 intended direction, 0.1 each sideways) require averaging over next states in the Bellman backup; deterministic env code will under-estimate risk. Action masking for invalid moves (walls) should zero out Q updates for impossible actions rather than letting agent waste exploration on illegal steps. SARSA (on-policy TD) updates with $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$ where a' is the action actually taken; compare to Q-learning when environment is highly stochastic.

14 Practice Problems

Problem 1. With $\gamma = 0.95$, step penalty -1 , goal reward $+10$ at 5 steps optimal path, approximate optimal $Q(s_0, a_0)$.

Problem 2. Implement ϵ decay from 1.0 to 0.05 over 500 episodes; plot return vs episode.

Problem 3. Compare synchronous updates vs updating Q while stepping within the same episode.

Problem 4. Visualize greedy policy arrows on the grid after training.

Implement in **Lab 15**; neural scaling in **Module 16**; policy gradients in **Module 17**.

References

- [1] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.
- [2] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.
- [3] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8:279–292, 1992.
- [4] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. volume 8, pages 229–256, 1992.